

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К курсовому проектированию

По дисциплине «Логика и основы алгоритмизации в инженерных
задачах»

На тему «Реализация алгоритма Дейкстры»

Выполнил студент
группы 24ВВВ2:

Тремасов К. И.

Приняла:

к.т.н., доцент Юрова О.В.

Пенза, 2025

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К курсовому проектированию

По дисциплине «Логика и основы алгоритмизации в инженерных
задачах»

На тему «Реализация алгоритма Дейкстры»

Выполнил студент

группы 24ВВВ2:

Тремасов К. И.

Приняла:

к.т.н., доцент Юрова О.В.

23.12.25
отлично
ф.и.о.

Пенза, 2025

НИЖНИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет Вычислительной техники

Кафедра "Вычислительная техника"

[Подпись]

Зав. кафедрой ВТ _____

"УТВЕРЖДАЮ"

« _____ » _____ 20__

ЗАДАНИЕ

на курсовое проектирование по курсу

Студенту Андреасу Баранцу Группа 240102-2
Тема проекта Визуализация алгоритма Дейкстры

Исходные данные (технические требования) на проектирование

- Разработка алгоритма и программного обеспечения
в соответствии с заданием курсового
проекта. Разрабатываемая программа должна
выполнять:
1. Ввод данных задачи;
 2. Проверку корректности данных задачи;
 3. Вывод алгоритма поиска кратчайшего пути;
 4. Вывод результатов работы программы и вычислений
(на каждом этапе работы алгоритма);
 5. Вывод схемы графа;
 6. Вывод кратчайшего пути;
 7. Вывод времени работы программы.

Объем работы по курсу

1. Расчетная часть

Вручай расчет работы алгоритма

2. Графическая часть

Схема алгоритма в формате блок-схем

3. Экспериментальная часть

Тестирование программы;
Разработка работы программы на тестовых данных

Срок выполнения проекта по разделам

- 1 Исследование теоретической части курсовой
- 2 Разработка алгоритмов программы
- 3 Разработка программы
- 4 Тестирование и завершение разработки программы
- 5 Оформление пояснительной записки
- 6
- 7
- 8

Дата выдачи задания "06" сентября

Дата защиты проекта " "

Руководитель Евдокимов Игорь О.В.

Задание получил "06" сентября 2025г.

Студент Мухомов Кирилл Владимирович

Содержание

Реферат	6
Введение.....	7
1. Постановка задачи	9
2. Теоретическая часть задания	10
3. Описание алгоритма	11
3.1 Алгоритм Дейкстры.....	11
3.2 Алгоритм ввода данных	12
4. Описание программы.....	14
4.1 Функция main	15
4.2 Функция manualInput.....	15
4.3 Функция generateRandomGraph	16
4.4 Функция displayGraph.....	16
4.5 Функция saveGraphToFile	16
4.6 Функция dijkstra	17
4.7 Функция appendResultsToFile	17
5. Тестирование	19
6. Ручной расчёт	25
Заключение	27
Список литературы	28
Листинг программы	29

Реферат

Отчёт 31 страницы, 8 рисунков, 2 таблицы, 5 источника.

ГРАФ, ВЗВЕШЕННЫЙ ГРАФ, АЛГОРИТМ ДЕЙКСТРЫ, ПРОГРАММА, ВВОД ДАННЫХ, РАБОТА С ФАЙЛАМИ.

В данной курсовой работе рассматривается реализация алгоритма Дейкстры для нахождения кратчайших путей от одной вершины до всех остальных в взвешенном графе. Цель исследования — разработка программы, которая позволяет эффективно вычислять оптимальные расстояния и маршруты в графах с неотрицательными весами рёбер, а также наглядно визуализировать процесс работы алгоритма и полученные результаты. Актуальность задачи обусловлена широким применением алгоритма Дейкстры в таких областях, как маршрутизация в компьютерных и транспортных сетях, проектирование логистических систем, анализ графовых моделей данных и оптимизация процессов в телекоммуникациях.

В работе подробно исследуются теоретические основы алгоритма Дейкстры, который является классическим решением задачи поиска кратчайших путей, эффективно использующим структуры данных для приоритетной обработки вершин. Также особое внимание уделяется практической части реализации: выбору способов представления графа в памяти программы (матрица смежности, списки смежности), методам ввода исходных данных (ручное задание, загрузка из файла, генерация тестовых графов) и организации вывода результатов (табличное представление расстояний, восстановление путей, графическая иллюстрация графа с выделенными кратчайшими маршрутами). Разработанная программа предоставляет удобный инструмент для анализа взвешенных графов, демонстрирует высокую эффективность даже на графах большого размера и может служить основой для интеграции в более сложные программные комплексы, решающие задачи сетевого планирования и оптимизации.

Введение

Поиск оптимальных маршрутов во взвешенных графах представляет собой одну из ключевых задач теории графов и прикладной информатики, находящую применение в проектировании транспортных и компьютерных сетей, логистическом планировании, маршрутизации данных и анализе социально-экономических связей. Особое значение в рамках этой задачи имеет нахождение кратчайших путей от заданной вершины до всех остальных — проблема, для которой классическим и наиболее эффективным решением для графов с неотрицательными весами рёбер является алгоритм Дейкстры.

С точки зрения программной реализации алгоритм Дейкстры предъявляет строгие требования к выбору структур данных и управлению вычислительными ресурсами. Его эффективность напрямую зависит от корректного применения приоритетной очереди (чаще всего на основе двоичной кучи), обеспечивающей быстрый доступ к вершине с минимальным текущим расстоянием. Кроме того, существенную роль играет способ представления графа в памяти (матрица смежности, списки смежности), влияющий на производительность при обработке разреженных или плотных графов.

В рамках данной курсовой работы разрабатывается программа на языке C++, реализующая алгоритм Дейкстры для неориентированного взвешенного графа. Основное внимание уделяется корректной алгоритмической реализации, включая построение эффективного механизма обновления расстояний и восстановления самих путей. Программа предусматривает гибкий ввод данных: ручное задание рёбер, генерацию случайного графа с заданными параметрами, а также сохранение и загрузку результатов. Наглядный вывод включает отображение матрицы смежности, таблицы кратчайших расстояний и подробного описания маршрутов.

Язык C++ выбран благодаря своей производительности, наличию стандартных библиотек контейнеров (таких как `vector` и `priority_queue`), которые идеально подходят для реализации алгоритма, и полному контролю над управлением памятью. Среда разработки Visual Studio 2022 предоставляет мощные средства отладки и профилирования, что важно для анализа эффективности работы программы на графах различного размера и плотности.

Цель работы — разработка, программная реализация и практическое исследование алгоритма Дейкстры для нахождения кратчайших путей в неориентированном взвешенном графе, а также создание удобного консольного приложения, демонстрирующего работу алгоритма и предоставляющего инструменты для анализа его результатов.

1. Постановка задачи

Требуется разработать программу, реализующую алгоритм Дейкстры для нахождения кратчайших путей от заданной стартовой вершины до всех остальных в неориентированном взвешенном графе с неотрицательными весами рёбер.

Кратчайшим путём между двумя вершинами считается путь, сумма весов рёбер которого минимальна среди всех возможных путей. Алгоритм должен корректно вычислять длины кратчайших путей от стартовой вершины до всех достижимых вершин графа, а также восстанавливать сами эти маршруты.

Программа должна предоставлять пользователю следующие способы задания структуры графа, такие как интерактивный ручной ввод рёбер (номера вершин и вес) и автоматическая генерация случайного графа с заданными параметрами (количество вершин, максимальный вес).

Главной частью программы является реализация классического алгоритма Дейкстры с использованием приоритетной очереди (min-heap) для выбора следующей вершины с минимальным текущим расстоянием. Алгоритм последовательно обрабатывает вершины, релаксируя (уменьшая) расстояния до её соседей, если через текущую вершину найден более короткий путь.

Результатом работы программы должны быть: вывод на экран матрицы смежности заданного графа и таблица, содержащая для каждой вершины длину кратчайшего пути от стартовой вершины и сам путь в виде последовательности вершин. Визуализация результата в виде текстового описания путей.

Необходимо реализовать функцию сохранения полных результатов работы программы (исходный граф, стартовая вершина, вычисленные пути и расстояния) в текстовый файл с форматированием и временной меткой.

2. Теоретическая часть задания

Граф G задается множеством вершин $V=\{v_1, v_2, \dots, v_n\}$ и множеством ребер E , соединяющих пары вершин. В рамках данной работы рассматривается взвешенный неориентированный граф, то есть граф, у которого каждому ребру $e=(u,v)$ сопоставлен неотрицательный числовой вес $w(u,v)$, и наличие ребра (u,v) подразумевает наличие ребра (v,u) с тем же весом. Вес интерпретируется как стоимость, расстояние или время перемещения между вершинами.

Одним из способов представления графа в памяти компьютера является матрица смежности. Для графа с n вершинами это квадратная матрица A размера $n \times n$, где элемент $A[i][j]$ равен весу ребра, соединяющего вершины i и j . Если ребра между вершинами не существует, элемент обычно принимает специальное значение (например, ноль или бесконечность). Для неориентированного графа матрица смежности является симметричной относительно главной диагонали.

Алгоритм поддерживает два основных множества вершин: множество обработанных вершин, для которых кратчайшее расстояние от источника уже найдено и является окончательным, и множество необработанных вершин, для которых известно некоторое расстояние от источника (возможно, не минимальное).

Изначально расстояние до источника полагается равным 0, а до всех остальных вершин — бесконечности.

Алгоритм не обходит вершины в случайном порядке или подряд. На каждом шаге он выбирает ту вершину, до которой уже найден самый короткий путь из всех еще не обработанных.

2. Описание алгоритма

Алгоритм решает задачу нахождения кратчайших путей от одной заданной вершины графа до всех остальных. Он применяется к взвешенным графам, веса всех рёбер которого неотрицательны. Основная идея алгоритма заключается в последовательном уточнении кратчайшего расстояния до каждой вершины, начиная со стартовой.

На подготовительном этапе всем вершинам, кроме стартовой, присваивается условно бесконечное расстояние. Расстояние до стартовой вершины устанавливается равным нулю. Все вершины помечаются как непосещённые.

Алгоритм выполняется циклически, пока остаются непосещённые вершины, до которых можно добраться из стартовой. На каждом шаге из всех ещё не посещённых вершин выбирается вершина с наименьшим известным на данный момент расстоянием от начала. Эта вершина помечается как посещённая, и её расстояние считается окончательным и неизменным. Это ключевое свойство алгоритма, гарантированное неотрицательностью весов рёбер: если в данный момент вершина является ближайшей среди непосещённых, то путь до неё не может быть сокращён через другие, более удалённые вершины.

Работа завершается, когда все вершины, достижимые из стартовой, будут посещены и для них установлены окончательные кратчайшие расстояния.

3.1 Алгоритм Дейкстры

1. Создать массив `distance` размером `vertices`, заполнить бесконечностью (`INF`)
2. Создать массив `visited` размером `vertices`, заполнить `false`
3. Создать приоритетную очередь `pq`
4. `distance[startVertex] = 0`

5. Добавить в pq пару(0, startVertex)
6. Пока pq не пуста :
7. Извлечь из pq вершину u с минимальным distance
8. Если visited[u] равно true : перейти к следующей итерации(пропустить)
9. Пометить visited[u] = true
10. Для каждой вершины v от 0 до vertices - 1 :
 11. Если существует ребро из u в v(adjacencyMatrix[u][v] > 0)
И visited[v] равно false :
 12. Вычислить newDist = distance[u] + вес_ребра(u, v)
 13. Если newDist < distance[v] :
 14. distance[v] = newDist
 15. Добавить в pq пару(newDist, v)
16. Вернуть массив distance

3.2 Алгоритм ввода данных

3. Пока истина (бесконечный цикл):
4. Вывести "Введите ребро: "
5. Считать строку input
6. Если input равна "stop" или "STOP":
 7. Прервать цикл
8. Разбить строку input на три числа u, v, weight
9. Если разбить не удалось (ошибка формата):
 10. Вывести сообщение об ошибке
 11. Перейти к следующей итерации
12. Если u или v вне диапазона [0, vertices-1]:
 13. Вывести сообщение об ошибке диапазона

14. Перейти к следующей итерации
15. Если u равно v :
 16. Вывести сообщение об ошибке (петли запрещены)
 17. Перейти к следующей итерации
18. Если $weight \leq 0$:
 19. Вывести сообщение об ошибке веса
 20. Перейти к следующей итерации
21. Добавить ребро между u и v с весом $weight$
22. Вывести подтверждение: "Добавлено ребро $u - v$ (вес: $weight$)"

После запуска пользователю отображается главное меню, где он может выбрать действия: создать случайный граф, ввести граф вручную, загрузить граф из файла, вывести граф на экран, найти компоненты сильной связности, сохранить граф в файл, сохранить результат в файл, очистить память и выйти.

Результат работы программы выводится на экран и может быть сохранён в файл.

4. Описание программы

Программа предназначена для нахождения кратчайших путей от начальной вершины до остальных вершин в графе. Реализована в Visual Studio 2022.

Общая структура программы в виде диаграммы взаимодействия модулей, демонстрирующая связь между основными компонентами системы (Рисунок 1).

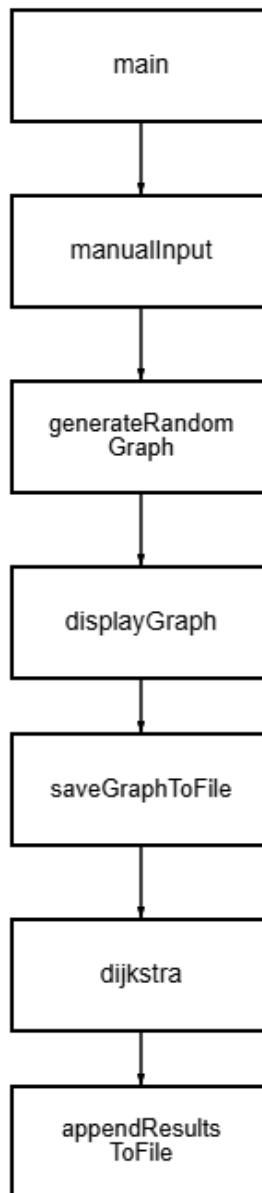


Рисунок 1 — структура модулей программы

4.1 Функция **main**

Назначение: Основная управляющая функция программы, которая координирует ввод данных, выполнение алгоритма Дейкстры и взаимодействие с пользователем.

Логика работы:

1. Запрашивает у пользователя количество вершин графа и способ ввода данных (ручной или случайная генерация).
2. Создаёт объект графа и инициирует построение графа согласно выбранному методу.
3. Выводит матрицу смежности на экран и сохраняет её в файл.
4. В цикле запрашивает стартовую вершину, выполняет алгоритм Дейкстры и сохраняет результаты.
5. Предлагает пользователю возможность выбора новой стартовой вершины или завершения программы.

4.2 Функция **manualInput**

Назначение: Обеспечивает интерактивный ручной ввод рёбер графа с весами от пользователя.

Логика работы:

1. Выводит инструкции по формату ввода: вершина1, вершина2, вес.
2. В цикле принимает строки ввода, анализирует их для извлечения данных о рёбрах.
3. Проверяет корректность введённых данных (диапазон вершин, положительный вес, отсутствие петель).
4. Добавляет корректные рёбра в матрицу смежности и подтверждает добавление.

4.3 Функция `generateRandomGraph`

Назначение: Генерирует случайный неориентированный взвешенный граф с заданной вероятностью наличия рёбер.

Логика работы:

1. Очищает существующую матрицу смежности.
2. Для каждой возможной пары вершин (i, j) , где $i < j$, с вероятностью 30% создаёт ребро.
3. Назначает случайный вес от 1 до заданного максимума для созданного ребра.
4. Заполняет обе ячейки матрицы смежности (i, j) и (j, i) для обеспечения неориентированности.

4.4 Функция `displayGraph`

Назначение: Визуализирует матрицу смежности графа в удобочитаемом табличном формате в консоли.

Логика работы:

1. Выводит заголовок и номера вершин в качестве заголовков столбцов.
2. Для каждой строки (вершины) выводит её номер и соответствующие веса рёбер.
3. Заменяет нулевые значения (отсутствие рёбер) на прочерки для улучшения читаемости.
4. Форматирует вывод с использованием манипуляторов для выравнивания столбцов.

4.5 Функция `saveInitialInfoToFile`

Назначение: Визуализирует матрицу смежности графа в удобочитаемом табличном формате в консоли.

Логика работы:

1. Создаёт файл.

2. Записывает заголовок с временной меткой и количеством вершин.
3. Преобразует матрицу смежности в строковый формат и записывает её в файл.
4. Закрывает файл, оставляя его готовым для последующего добавления результатов вычислений.

4.6 Функция `dijkstra`

Назначение: Вычисляет кратчайшие расстояния от заданной стартовой вершины до всех остальных вершин графа.

Логика работы:

1. Инициализирует массив расстояний значением INF (бесконечность) и массив посещённых вершин.
2. Устанавливает расстояние до стартовой вершины равным 0 и помещает её в очередь с приоритетами.
3. Извлекает вершину с минимальным расстоянием, помечает её как посещённую.
4. Для всех смежных непосещённых вершин обновляет расстояния, если найден более короткий путь.
5. Повторяет процесс, пока очередь не станет пустой, возвращая массив кратчайших расстояний.

4.7 Функция `appendResultsToFile`

Назначение: Добавляет результаты выполнения алгоритма Дейкстры для конкретной стартовой вершины в существующий файл.

Логика работы:

1. Открывает файл "path.txt" в режиме добавления (не перезаписывая существующее содержимое).
2. Форматирует результаты: кратчайшие пути, расстояния и статистику для текущей стартовой вершины.

3. Для каждой вершины графа выводит расстояние и полный путь (если он существует).

4. Вычисляет и записывает статистику: количество достижимых вершин, среднее расстояние и т.д.

5. Разделяет результаты разных запусков визуальными разделителями для удобства чтения.

5. Тестирование

Для разработки и отладки программы использовалась интегрированная среда Microsoft Visual Studio 2022, предоставляющая полный набор инструментов для создания, компиляции и тестирования многомодульных консольных приложений.

Тестирование программы будет включать проверку корректности работы ядра алгоритма на графах различной корректности загрузки и сохранения данных в файл, обработки граничных случаев (пустой граф, граф с одной вершиной), а также устойчивости к потенциальным ошибочным ситуациям (неверный формат входного файла, попытка обработки неинициализированного графа). В таблице 1 представлено описание запланированных тестов программы.

Таблица 1 — тестирование функционала программы

№ теста	Проводимые действия	Алгоритм	Ожидаемый результат
1	Запуск исполняемого файла	Вызов функции main()	Вывод меню действий
2	Ввод 4 вершин в графе	Определение количества вершин	Создан пустой граф с 4 вершинами
3	Ручной ввод связей вершин 0-4	Заполнение матрицы смежности вручную	Взвешенный неориентированный граф
4	Генерация ориентированного графа и ввод максимального веса для случайных связей	Создание случайного ориентированного графа	Взвешенный ориентированный граф
5	Генерация неориентированного графа и ввод максимального веса для случайных связей	Создание случайного неориентированного графа	Взвешенный неориентированный граф
6	Ввод начальной вершины	Поиск расстояний до вершин (Дейкстры)	Расстояния от начальной вершины до всех вершин графа

При запуске программы пользователь вводит количество вершин графа, после ввода он видит меню с двумя вариантами: ручной ввод и генерация случайного графа (Рисунок 2).

```
===== DIJKSTRA'S ALGORITHM =====  
===== Tremasov Kirill, 24VVV2 =====  
  
Enter number of vertices: 4  
  
Choose input method:  
1. Manual input  
2. Generate random graph  
Enter choice (1 or 2):
```

Рисунок 2 — начало работы программы

При выборе первого варианта 1 выводится инструкция (Рисунок 3), и пользователь вводит связи для всех вершин в любом порядке (Рисунок 4).

```
Enter edges in format: (vertex1) (vertex2) (weight)  
Example: 0 1 5 (creates edge between vertex 0 and 1 with weight 5)  
Enter 'stop' to finish input  
Note: Vertex numbers should be between 0 and 3
```

Рисунок 3 — инструкция по вводу связей

```
Enter edge: 0 1 5  
Edge added: 0 - 1 (weight: 5)  
Enter edge: 1 0 5  
Edge added: 1 - 0 (weight: 5)  
Enter edge: 0 3 3  
Edge added: 0 - 3 (weight: 3)  
Enter edge: 3 0 3  
Edge added: 3 - 0 (weight: 3)  
Enter edge: 1 3 2  
Edge added: 1 - 3 (weight: 2)  
Enter edge: 3 1 2  
Edge added: 3 - 1 (weight: 2)  
Enter edge: 1 2 1  
Edge added: 1 - 2 (weight: 1)  
Enter edge: 2 1 1  
Edge added: 2 - 1 (weight: 1)
```

Рисунок 4 — ввод связей

После того как были введены все связи пользователь вводит команду stop, и матрица выводится на экран (Рисунок 5).

```
Enter edge: stop
Edge input completed.

Adjacency Matrix:
      0  1  2  3
-----
0 |  0  5  -  3
1 |  5  0  1  2
2 |  -  1  0  -
3 |  3  2  -  0

Graph information has been saved to path.txt
```

Рисунок 5 — матрица смежности

Пользователь вводит варианта 2, выбирает ориентированный граф и вводит максимально возможный вес связи и программа генерирует таблицу (Рисунок 6). Можно выбрать и неориентированный граф (Рисунок 7).

```
Choose input method:
1. Manual input
2. Generate random graph
Enter choice (1 or 2): 2

Choose graph type:
1. Directed graph
2. Undirected graph
Enter choice (1 or 2): 1
Enter maximum edge weight: 100
Random directed graph generated.

Adjacency Matrix (Directed Graph):
      0  1  2  3
-----
0 |  -  53  33  70
1 |  89  -  -  91
2 |  -  61  -  73
3 |  -  -  -  -

Enter filename to save results: _
```

Рисунок 6 — случайная матрица смежности (ориентированный граф)

```

===== DIJKSTRA'S ALGORITHM =====
===== Tremasov Kirill, 24VVV2 =====

Enter number of vertices: 4

Choose input method:
1. Manual input
2. Generate random graph
Enter choice (1 or 2): 2

Choose graph type:
1. Directed graph
2. Undirected graph
Enter choice (1 or 2): 2
Enter maximum edge weight: 100
Random undirected graph generated.

Adjacency Matrix (Undirected Graph):
      0   1   2   3
-----
0 |   -   -   -  91
1 |   -   -   -  32
2 |   -   -   -   7
3 |  91  32   7   -

Enter filename to save results:

```

Рисунок 7 — случайная матрица смежности (неидентифицированный граф)

Далее выбираем вершину. Алгоритм Дейкстры находит все кратчайшие пути для вершин. Сохраняем результаты в файл path.txt. Также можно перевыбрать стартовую вершину (Рисунок 8).

```

Shortest paths from vertex 0:
Vertex Distance Path
0 0 0
1 5 0 -> 1
2 6 0 -> 1 -> 2
3 3 0 -> 3
Results for vertex 0 have been added to path.txt

Do you want to choose another starting vertex? (y/n): y

Enter starting vertex (0 to 3): 2

Shortest paths from vertex 2:
Vertex Distance Path
0 6 2 -> 1 -> 0
1 1 2 -> 1
2 0 2
3 3 2 -> 1 -> 3
Results for vertex 2 have been added to path.txt

Do you want to choose another starting vertex? (y/n): _

```

Рисунок 8 — Результат работы алгоритма

При отказе выбрать следующую начальную вершину программа завершается (Рисунок 9).

```

Do you want to choose another starting vertex? (y/n): n

Program completed successfully.
All results have been saved to path.txt

Press Enter to exit...

C:\Users\andor\source\repos\kursProject\x64\Debug\kursPro
Нажмите любую клавишу, чтобы закрыть это окно:

```

Рисунок 9 — завершение работы программы

В таблице приведены результаты тестирования (Таблица 2).

Таблица 2 — результаты тестирования

№ теста	Проводимые действия	Алгоритм	Полученный результат
1	Запуск исполняемого файла	Вызов функции main()	Верно
2	Ввод 4 вершин в графе	Определение количества вершин	Верно
3	Ручной ввод связей вершин 0-4	Заполнение матрицы смежности вручную	Верно

Продолжение Таблицы 2.

4	Генерация ориентированного графа и ввод максимального веса для случайных связей	Создание случайного ориентированного графа	Верно
5	Генерация неориентированного графа и ввод максимального веса для случайных связей	Создание случайного неориентированного графа	Верно
4	Выбор 2: ввод максимального веса для случайных связей	Создание случайного графа	Верно
5	Ввод начальной вершины	Поиск расстояний до вершин алгоритмом Дейкстры	Верно

6. Ручной расчёт

Ручной расчёт кратчайших путей подтверждает, что программа работает корректно и выдаёт правильный результат (Рисунок 10). Ручной расчет сравнили с результатом работы программы (Рисунок 11).

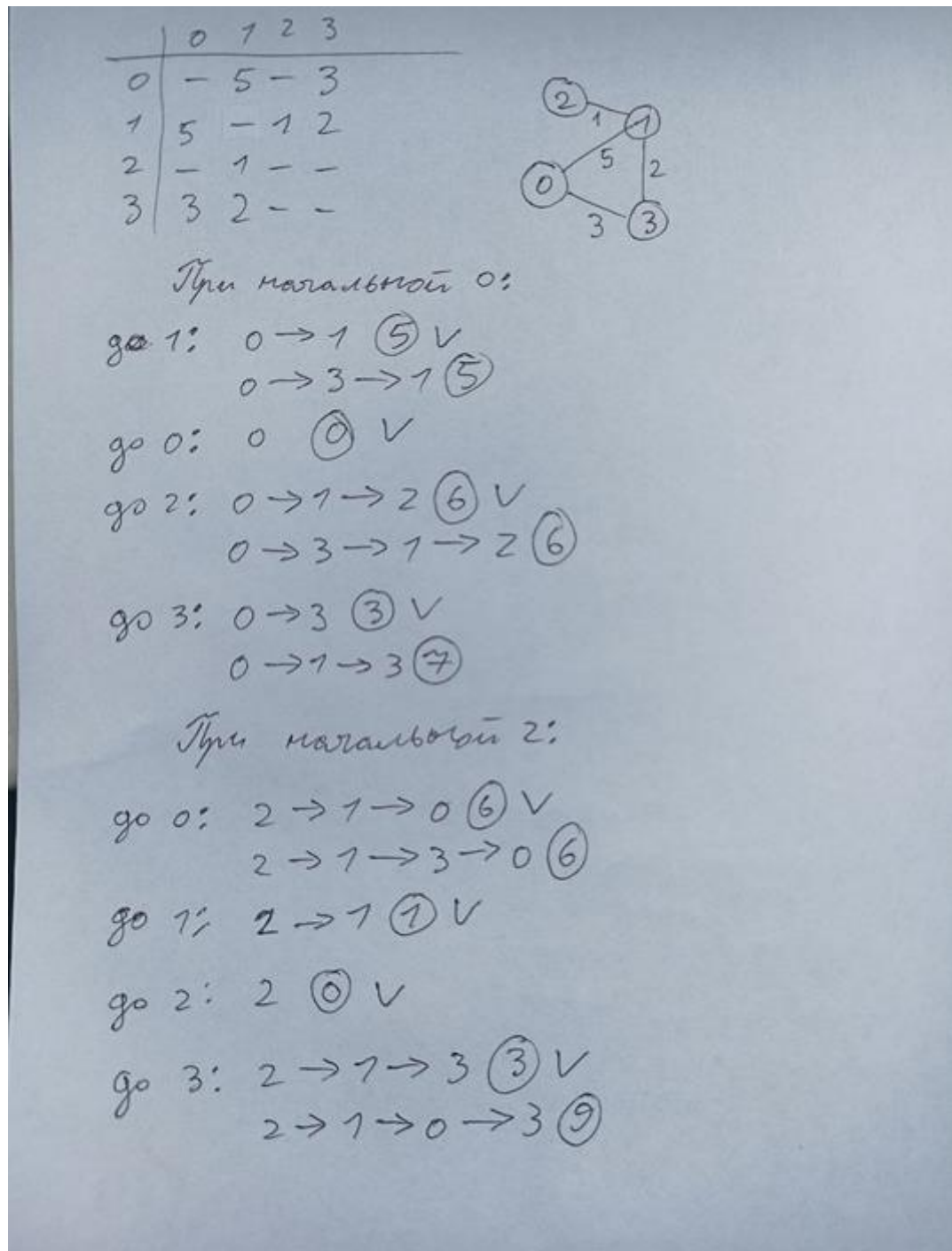


Рисунок 10 — ручной расчет

```

Adjacency Matrix (Undirected Graph):
      0   1   2   3
-----
0 |   -   5   -   3
1 |   5   -   1   2
2 |   -   1   -   -
3 |   3   2   -   -

Enter filename to save results: a.txt
Results will be saved to: .txt

Graph information has been saved to .txt

Enter starting vertex (0 to 3): 0

Shortest paths from vertex 0:
Vertex Distance      Path
0         0           0
1         5           0 -> 1
2         6           0 -> 1 -> 2
3         3           0 -> 3

```

Рисунок 11 — проверка речетов

Заключение

В ходе выполнения данной курсовой работы была успешно разработана и реализована программа для нахождения кратчайших путей во взвешенном графе с использованием алгоритма Дейкстры. Теоретическое исследование подтвердило обоснованность выбора данного алгоритма как классического, эффективного и гарантированно корректного решения для графов с неотрицательными весами ребер. Практическая реализация на языке C++ продемонстрировала применение ключевых алгоритмических принципов — жадной стратегии выбора вершин, использования приоритетной очереди для оптимизации и работы с фундаментальными структурами данных (матрицей смежности), что в полной мере соответствует целям изучения дисциплины «Программирование и алгоритмизация».

Разработанное программное обеспечение обладает завершенной функциональностью: оно предоставляет пользователю удобные способы задания графа (интерактивный ввод и автоматическая генерация), надежно выполняет вычислительное ядро — алгоритм Дейкстры — и обеспечивает комплексный вывод результатов как на экран, так и в структурированный текстовый файл с возможностью последующего анализа.

Цель работы достигнута — создан корректно функционирующий инструмент для решения задачи поиска кратчайших путей. Реализованный проект служит не только практическим подтверждением освоения теоретических основ алгоритмов на графах, но и составляет готовую основу для интеграции в более сложные программные комплексы, предназначенные для решения прикладных задач в областях сетевой маршрутизации, логистического планирования и анализа транспортных систем.

Список литературы

1. Кормен, Т., Лейзерсон, Ч., Ривест, Р., Штайн, К. Алгоритмы: построение и анализ. – 3-е изд. – М.: Вильямс, 2022. – 1328 с. – (Классический фундаментальный учебник, содержащий строгое формальное описание алгоритма Дейкстры, доказательство его корректности и детальный анализ временной сложности с использованием различных структур данных).
2. Седжвик, Р., Уэйн, К. Алгоритмы на Java. – 4-е изд. – М.: Вильямс, 2020. – 848 с. – (Практическое руководство с акцентом на эффективные реализации, включающее подробный разбор алгоритмов на графах, в том числе алгоритма Дейкстры, с использованием приоритетных очередей).
3. Ахо, А.В., Хопкрофт, Дж.Э., Ульман, Дж.Д. Структуры данных и алгоритмы. – М.: Диалектика, 2020. – 400 с. – (Фундаментальная работа, подробно рассматривающая теоретические основы алгоритмов поиска кратчайших путей и абстрактные структуры данных, необходимые для их реализации).
4. Окулов, С.М. Программирование в алгоритмах. – 6-е изд., перераб. и доп. – М.: БИНОМ. Лаборатория знаний, 2022. – 383 с. – (Содержит практические рекомендации и примеры эффективных реализаций на C/C++ классических алгоритмов, включая алгоритм Дейкстры, с анализом типичных ошибок и способов оптимизации).
5. Скиена, С.С. Алгоритмы. Руководство по разработке. – 3-е изд. – М.: Диалектика, 2021. – 720 с. – (Практико-ориентированное издание, в котором задача поиска кратчайшего пути рассматривается в контексте проектирования программных систем, с примерами кода и сравнительным анализом алгоритмов).

Листинг программы

```
#include <iostream>
#include <vector>
#include <limits>
#include <random>
#include <iomanip>
#include <queue>
#include <utility>
#include <sstream>
#include <string>
#include <fstream>
#include <ctime>

using namespace std;

const int INF = numeric_limits<int>::max();

class Graph {
private:
    int vertices;
    bool directed;
    vector<vector<int>> adjacencyMatrix;

    bool isValidVertex(int v) {
        return v >= 0 && v < vertices;
    }

public:
    Graph(int n, bool dir = false) : vertices(n), directed(dir), adjacencyMatrix(n,
vector<int>(n, 0)) {}

    void addEdge(int u, int v, int weight, bool isDirected = false) {
        if (isValidVertex(u) && isValidVertex(v) && u != v) {
            adjacencyMatrix[u][v] = weight;
            if (!isDirected && !directed) {
                adjacencyMatrix[v][u] = weight;
            }
        }
    }

    void generateRandomGraph(int maxWeight, bool isDirected = false) {
        directed = isDirected;

        random_device rd;
        mt19937 gen(rd());
        uniform_int_distribution<> dist(1, maxWeight);
        uniform_real_distribution<> probDist(0.0, 1.0);

        for (int i = 0; i < vertices; i++) {
            for (int j = 0; j < vertices; j++) {
                adjacencyMatrix[i][j] = 0;
            }
        }

        if (isDirected) {
            for (int i = 0; i < vertices; i++) {
                for (int j = 0; j < vertices; j++) {
                    if (i != j && probDist(gen) <= 0.4) {
```

```

        int weight = dist(gen);
        adjacencyMatrix[i][j] = weight;
    }
}
}
else {
    for (int i = 0; i < vertices; i++) {
        for (int j = i + 1; j < vertices; j++) {
            if (probDist(gen) <= 0.7) {
                int weight = dist(gen);
                adjacencyMatrix[i][j] = weight;
                adjacencyMatrix[j][i] = weight;
            }
        }
    }
}

void displayGraph() {
    cout << "\nAdjacency Matrix (" << (directed ? "Directed" : "Undirected") << "
Graph):\n";
    cout << "      ";
    for (int i = 0; i < vertices; i++) {
        cout << setw(4) << i;
    }
    cout << "\n";
    cout << "      ";
    for (int i = 0; i < vertices; i++) {
        cout << "----";
    }
    cout << "\n";

    for (int i = 0; i < vertices; i++) {
        cout << setw(3) << i << " |";
        for (int j = 0; j < vertices; j++) {
            if (adjacencyMatrix[i][j] == 0) {
                cout << setw(4) << "-";
            }
            else {
                cout << setw(4) << adjacencyMatrix[i][j];
            }
        }
        cout << "\n";
    }
}

string getAdjacencyMatrixString() {
    stringstream ss;
    ss << "Adjacency Matrix (" << (directed ? "Directed" : "Undirected") << "
Graph):\n";
    ss << "      ";
    for (int i = 0; i < vertices; i++) {
        ss << setw(4) << i;
    }
    ss << "\n";
    ss << "      ";
    for (int i = 0; i < vertices; i++) {
        ss << "----";
    }
    ss << "\n";
}

```

```

    for (int i = 0; i < vertices; i++) {
        ss << setw(3) << i << " |";
        for (int j = 0; j < vertices; j++) {
            if (adjacencyMatrix[i][j] == 0 && i != j) {
                ss << setw(4) << "-";
            }
            else {
                ss << setw(4) << adjacencyMatrix[i][j];
            }
        }
        ss << "\n";
    }
    return ss.str();
}

vector<int> dijkstra(int startVertex) {
    vector<int> distance(vertices, INF);
    vector<bool> visited(vertices, false);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int,
int>>> pq;

    distance[startVertex] = 0;
    pq.push({ 0, startVertex });

    while (!pq.empty()) {
        int u = pq.top().second;
        pq.pop();

        if (visited[u]) continue;
        visited[u] = true;

        for (int v = 0; v < vertices; v++) {
            if (adjacencyMatrix[u][v] > 0 && !visited[v]) {
                int newDist = distance[u] + adjacencyMatrix[u][v];
                if (newDist < distance[v]) {
                    distance[v] = newDist;
                    pq.push({ newDist, v });
                }
            }
        }
    }

    return distance;
}

void displayShortestPaths(int startVertex, const vector<int>& distances) {
    cout << "\nShortest paths from vertex " << startVertex << ":\n";
    cout << "Vertex\tDistance\tPath\n";
    for (int i = 0; i < vertices; i++) {
        cout << i << "\t";
        if (distances[i] == INF) {
            cout << "INF\t\tNo path";
        }
        else {
            cout << distances[i] << "\t\t";
            vector<int> path = getPath(startVertex, i, distances);
            for (size_t j = 0; j < path.size(); j++) {
                cout << path[j];
                if (j != path.size() - 1) {
                    cout << " -> ";
                }
            }
        }
    }
}

```

```

        if (path.empty()) {
            cout << startVertex;
        }
    }
    cout << "\n";
}

string getShortestPathsString(int startVertex, const vector<int>& distances) {
    stringstream ss;
    ss << "\n===== \n";
    ss << "SHORTEST PATHS FROM VERTEX " << startVertex << ":\n";
    ss << "===== \n";
    ss << "Vertex\tDistance\tPath\n";
    ss << "----- \n";

    for (int i = 0; i < vertices; i++) {
        ss << i << "\t";
        if (distances[i] == INF) {
            ss << "INF\t\tNo path";
        }
        else {
            ss << distances[i] << "\t\t";
            vector<int> path = getPath(startVertex, i, distances);
            if (path.empty() && i == startVertex) {
                ss << startVertex;
            }
            else {
                for (size_t j = 0; j < path.size(); j++) {
                    ss << path[j];
                    if (j != path.size() - 1) {
                        ss << " -> ";
                    }
                }
            }
        }
        ss << "\n";
    }

    ss << "\n===== \n";
    ss << "SUMMARY FOR START VERTEX " << startVertex << ":\n";
    ss << "===== \n";
    int reachableCount = 0;
    int totalDistance = 0;

    for (int i = 0; i < vertices; i++) {
        if (distances[i] != INF && i != startVertex) {
            reachableCount++;
            totalDistance += distances[i];
        }
    }

    ss << "Total vertices: " << vertices << "\n";
    ss << "Graph type: " << (directed ? "Directed" : "Undirected") << "\n";
    ss << "Reachable vertices from " << startVertex << ": " << reachableCount <<
"\n";
    ss << "Unreachable vertices: " << (vertices - 1 - reachableCount) << "\n";
    if (reachableCount > 0) {
        ss << "Average distance to reachable vertices: "
<< fixed << setprecision(2) << (double)totalDistance / reachableCount
<< "\n";
    }
}

```



```

        ss << "=====\n\n";

        return ss.str();
    }

    bool saveInitialInfoToFile(const string& filename) {
        ofstream outFile(filename);

        if (!outFile.is_open()) {
            cerr << "Error: Could not open file " << filename << " for writing.\n";
            return false;
        }

        time_t now = time(0);
        char timeStr[26];

#ifdef _WIN32
        if (ctime_s(timeStr, sizeof(timeStr), &now) != 0) {
            strcpy_s(timeStr, sizeof(timeStr), "Unknown time");
        }
#else
        ctime_r(&now, timeStr);
#endif

        size_t len = strlen(timeStr);
        if (len > 0 && timeStr[len - 1] == '\n') {
            timeStr[len - 1] = '\0';
        }

        outFile << "==== DIJKSTRA'S ALGORITHM RESULTS =====\n";
        outFile << "Generated: " << timeStr << "\n";
        outFile << "Number of vertices: " << vertices << "\n";
        outFile << "Graph type: " << (directed ? "Directed" : "Undirected") << "\n";
        outFile << "Output file: " << filename << "\n";
        outFile << "=====\n\n";

        outFile << getAdjacencyMatrixString();
        outFile << "\n=====\n\n";

        outFile.close();
        return true;
    }

    bool appendResultsToFile(const string& filename, int startVertex, const
vector<int>& distances) {
        ofstream outFile(filename, ios::app);

        if (!outFile.is_open()) {
            cerr << "Error: Could not open file " << filename << " for appending.\n";
            return false;
        }

        outFile << getShortestPathsString(startVertex, distances);
        outFile.close();
        return true;
    }

    vector<int> getPath(int start, int end, const vector<int>& distances) {
        vector<int> path;
        if (start == end) {
            path.push_back(start);
            return path;
        }
    }

```

```

    }

    if (distances[end] == INF) return path;

    int current = end;
    path.push_back(current);

    while (current != start) {
        bool found = false;
        for (int i = 0; i < vertices; i++) {
            if (adjacencyMatrix[i][current] > 0 &&
                distances[current] == distances[i] + adjacencyMatrix[i][current])
            {
                current = i;
                path.push_back(current);
                found = true;
                break;
            }
        }
        if (!found) {
            path.clear();
            path.push_back(start);
            if (start != end && adjacencyMatrix[start][end] > 0) {
                path.push_back(end);
            }
            return path;
        }
    }

    reverse(path.begin(), path.end());
    return path;
}

void manualInput() {
    cout << "\nEnter edges in format: (vertex1) (vertex2) (weight)\n";
    cout << "Example: 0 1 5   (creates edge between vertex 0 and 1 with weight
5)\n";
    if (directed) {
        cout << "Note: This is a DIRECTED graph. Edge will be from vertex1 to
vertex2\n";
    }
    else {
        cout << "Note: This is an UNDIRECTED graph. Edge will be added in both
directions\n";
    }
    cout << "Enter 'stop' to finish input\n";
    cout << "Note: Vertex numbers should be between 0 and " << vertices - 1 <<
"\n\n";

    string input;
    cin.ignore();

    while (true) {
        cout << "Enter edge: ";
        getline(cin, input);

        if (input == "stop" || input == "STOP") {
            break;
        }

        stringstream ss(input);
        int u, v, weight;

```

```

        if (ss >> u >> v >> weight) {
            string extra;
            if (ss >> extra) {
                cout << "Error: Too many values. Please enter exactly 3
numbers.\n";
                continue;
            }

            if (!isValidVertex(u) || !isValidVertex(v)) {
                cout << "Error: Vertices must be between 0 and " << vertices - 1
<< "\n";
                continue;
            }

            if (u == v) {
                cout << "Error: Vertices cannot be the same (no self-loops).\n";
                continue;
            }

            if (weight <= 0) {
                cout << "Error: Weight must be a positive integer.\n";
                continue;
            }

            addEdge(u, v, weight, directed);
            cout << "Edge added: " << u << (directed ? " -> " : " - ") << v << "
(weight: " << weight << ")\n";
        }
        else {
            cout << "Error: Invalid input format. Please enter 3 numbers separated
by spaces.\n";
            cout << "Example: 0 1 10\n";
        }
    }
    cout << "Edge input completed.\n";
}

bool isDirected() const {
    return directed;
}

};

int main() {
    int choice;
    int vertices;
    int startVertex;
    string filename;

    cout << "==== DIJKSTRA'S ALGORITHM ==== \n";
    cout << "==== Tremasov Kirill, 24VVV2 ==== \n";
    cout << "\n";

    cout << "Enter number of vertices: ";
    cin >> vertices;

    if (vertices <= 0) {
        cout << "Error: Number of vertices must be positive! \n";
        return 1;
    }

    bool isDirected = false;

```

```

cout << "\nChoose input method:\n";
cout << "1. Manual input\n";
cout << "2. Generate random graph\n";
cout << "Enter choice (1 or 2): ";
cin >> choice;

if (choice == 1) {
    cout << "\nChoose graph type for manual input:\n";
    cout << "1. Directed graph\n";
    cout << "2. Undirected graph\n";
    cout << "Enter choice (1 or 2): ";
    int graphType;
    cin >> graphType;

    if (graphType == 1) {
        isDirected = true;
        cout << "Selected: Directed graph\n";
    }
    else if (graphType == 2) {
        isDirected = false;
        cout << "Selected: Undirected graph\n";
    }
    else {
        cout << "Error: Invalid choice! Defaulting to undirected graph.\n";
        isDirected = false;
    }
}

Graph graph(vertices, isDirected);

if (choice == 1) {
    graph.manualInput();
}
else if (choice == 2) {
    int graphType;
    cout << "\nChoose graph type (undirected by default):\n";
    cout << "1. Directed graph\n";
    cout << "2. Undirected graph\n";
    cout << "Enter choice (1 or 2): ";
    cin >> graphType;

    bool generateDirected = (graphType == 1);

    int maxWeight;
    cout << "Enter maximum edge weight: ";
    cin >> maxWeight;
    if (maxWeight <= 0) {
        cout << "Error: Maximum weight must be positive!\n";
        return 1;
    }
    graph.generateRandomGraph(maxWeight, generateDirected);
    cout << "Random " << (generateDirected ? "directed" : "undirected") << " graph
generated.\n";
}
else {
    cout << "Error: Invalid choice!\n";
    return 1;
}

graph.displayGraph();

cout << "\nEnter filename to save results: ";

```

```

cin.ignore();
getline(cin, filename);

cout << "Results will be saved to: " << filename << endl;

bool fileInitialized = graph.saveInitialInfoToFile(filename);
if (!fileInitialized) {
    cout << "Warning: Could not create file. Results will not be saved.\n";
}
else {
    cout << "\nGraph information has been saved to " << filename << "\n";
}

char continueChoice = 'y';

while (continueChoice == 'y' || continueChoice == 'Y') {
    cout << "\nEnter starting vertex (0 to " << vertices - 1 << "): ";
    cin >> startVertex;

    if (startVertex < 0 || startVertex >= vertices || !isdigit(startVertex)) {
        cout << "Error: Invalid starting vertex!\n";
        break;
    }
    else if (startVertex >= 0) {
        vector<int> distances = graph.dijkstra(startVertex);
        graph.displayShortestPaths(startVertex, distances);

        if (fileInitialized) {
            bool appended = graph.appendResultsToFile(filename, startVertex,
distances);
            if (appended) {
                cout << "Results for vertex " << startVertex << " have been added
to " << filename << "\n";
            }
        }
    }

    cout << "\nDo you want to choose another starting vertex? (y/n): ";
    cin >> continueChoice;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

if (fileInitialized) {
    cout << "All results have been saved to " << filename << "\n";
}

cout << "\nPress Enter to exit...";
cin.get();

return 0;
}

```